

Dragon Hatchling (BDH)

↳ connects Transformers & brain

↳ works like graph of neurons
↳ performs like the transformer

(1) Logical reasoning

(2) Hebbian Learning

↳ strengthen connection b/w them

State-Space + Graph

Transformers

- struggle with complex tasks

- long reasoning/long time generalization

BDH — Architecture Overview

"hello"

encoded as bytes: [104, 101, 108, 108, 111] (no tokenizer needed)

[B, T] integer indices

Token Embedding

each of 256 possible bytes → a learnable vector of D=256 numbers

[B, T, D=256]

BDH Layer × n_layer (same weights reused every iteration)

① Expand

D=256 → N=8192
per head

② ReLU (sparse!)

~95% neurons
→ exactly 0

③ Linear Attention

tokens look at
past tokens (O(T))

④ Hebbian Gate ×

$x_{\text{sparse}} \times y_{\text{sparse}}$
"fire together"

⑤ compress back: $N \times \text{heads} \rightarrow D=256$ then $\text{output} + \text{input}$ (residual)

[B, T, D=256]

Output Projection

D=256 → 256 logits (one score per possible next byte)

softmax → probabilities

Sample next byte → append → repeat

Here each character = token 1) each character
= byte index
(ie. - $n \rightarrow 104$)

2) embedding lookup table

0
:
256

key	embedding vector
0	[x, - - -]
:	[- - -]
:	[- - -]
256	[- - -]

3) layer norm $\rightarrow \ln(\text{vector})$
(rescales the values mean = 0, std = 1
keeps training stable)

4) Expand to neuron space

vector @ encoder

5) Apply ReLU

(remove negative values)

(sparse positive activation)

$$x_{\text{sparse}} = \text{ReLU}(x @ \text{encoder})$$

How is positional info preserved while doing attention (aka comparing prev tokens)?

RoPE

- Ok let's trace through "hello" step by step.

We have 5 tokens, each at a position:

'h' → position 0
 'e' → position 1
 'l' → position 2
 'l' → position 3
 'o' → position 4

After embedding + expand + ReLU, each token has a sparse vector:

$x_sparse[0] = [0.0, 0.4, 0.0, 0.7, 0.0, \dots]$ ← 'h' at pos 0
 $x_sparse[1] = [0.0, 0.1, 0.3, 0.0, 0.0, \dots]$ ← 'e' at pos 1
 $x_sparse[2] = [0.5, 0.0, 0.0, 0.2, 0.0, \dots]$ ← 'l' at pos 2
 $x_sparse[3] = [0.5, 0.0, 0.0, 0.2, 0.0, \dots]$ ← 'l' at pos 3 (same content!)
 $x_sparse[4] = [0.0, 0.3, 0.0, 0.0, 0.6, \dots]$ ← 'o' at pos 4

Problem – positions 2 and 3 are identical:

$x_sparse[2] @ x_sparse[3] = \text{big number}$ (they're the same vector!)
 $x_sparse[2] @ x_sparse[2] = \text{same big number}$

The model can't tell "am I looking at the token right next to me, or myself?"

RoPE fix – rotate each vector by its position before the dot product:

$\text{rotate}(x_sparse[2], \text{angle}=2\theta)$ ← 'l' at pos 2, rotated a little
 $\text{rotate}(x_sparse[3], \text{angle}=3\theta)$ ← 'l' at pos 3, rotated slightly more

Now their dot product is smaller than if you compared position 2 with itself, because the angles are different. Two tokens far apart get rotated by very different amounts → their dot product shrinks → they attend to each other less.

$$\theta_{(pos,i)} = \frac{\text{pos}}{1000^{2i/0}}$$

ie--
 To rotate 256 D vectors,
 we break it into
 128 pairs of 2D vectors

this is what 2D rotation is

$$\begin{bmatrix} a \\ b \end{bmatrix} \mapsto \begin{bmatrix} a \cos \theta - b \sin \theta \\ a \sin \theta + b \cos \theta \end{bmatrix}$$

$\theta(\text{pos}, i) = \text{pos} / (10000^{(2i / 256)})$

Let's compute some actual values:

pair 0 (i=0): base = $10000^{(0/256)} = 1.0$
 pair 1 (i=1): base = $10000^{(2/256)} = 1.33$
 pair 2 (i=2): base = $10000^{(4/256)} = 1.78$
 ...
 pair 127 (i=127): base = $10000^{(254/256)} \approx 9966$

θ for 'h' (pos=0): $0/1.0=0, 0/1.33=0, 0/1.78=0 \dots$ all zero (no rotation)
 θ for 'e' (pos=1): $1/1.0=1, 1/1.33=0.75, 1/1.78=0.56 \dots$
 θ for 'l' (pos=2): $2/1.0=2, 2/1.33=1.5, 2/1.78=1.12 \dots$
 θ for 'l' (pos=3): $3/1.0=3, 3/1.33=2.25, 3/1.78=1.69 \dots$

Early pairs rotate **fast** (large angle). Later pairs rotate **slowly** (tiny angle). This is intentional – it creates a unique "fingerprint" for every position.

Just like in transformers, calculate and compute Q, K attention pattern

In BDH,
 $Q = K = \text{rotate (x-sparse, positions)}$

$$\text{scores} = Q @ K^T$$

$$S \times N @ N \times S = S \times S$$

Scores.trill (diagonal = -1)

remove the influence of tokens on the future ones
 ✓✓ makes them 0

In BDH, a token cannot attend to itself

∴ In transformer, diagonal = 0

BDH Attention Pattern (Orientation A: Rows=Keys, Cols=Queries)

Sentence: "a fluffy blue creature roamed the verdant forest"

attend to →

		Q ₁	Q ₂	Q ₃	Q ₄	Q ₅	Q ₆	Q ₇	Q ₈
		a	fluffy	blue	crea	roam	the	verd	forest
K ₁	a	0	•	•	•	•	•	•	•
K ₂	fluffy	0	0	•	•	•	•	•	•
K ₃	blue	0	0	0	•	•	•	•	•
K ₄	creature	0	0	0	0	•	•	•	•
K ₅	roamed	0	0	0	0	0	•	•	•
K ₆	the	0	0	0	0	0	0	•	•
K ₇	verdant	0	0	0	0	0	0	0	•
K ₈	forest	0	0	0	0	0	0	0	0

← scores

• = can attend 0 = blocked

$$a_star = scores @ V$$

then we expand

$$y_sparse = ReLU(a_star @ encoder_v)$$

$$gate = x_sparse * y_sparse$$

then compress

$$output = gate @ decoder$$

$$x = x + output \text{ (residual connection)}$$

original x embedding from lookup table

$$logits = x @ lm_head$$

Full forward pass – input to output:

"hello" → [104, 101, 108, 108, 111]

embed.weight[idx]

param 1 ✓

x [5, 256]

repeat n_layer times

x_sparse = ReLU(x @ encoder)

param 2 ✓

Q = rotate(x_sparse)

(no params, just RoPE)

K = rotate(x_sparse)

(no params, reuses x)

V = x (raw embedding)

scores = Q @ K^T → causal mask

(no params)

a_star = scores @ V [5, 256]

(no params)

y_sparse = ReLU(a_star @ encoder_v)

param 3 ✓

gate = x_sparse * y_sparse

(no params)

output = gate @ decoder

param 4 ✓

x = x + output (residual)

(no params)

logits = x @ lm_head

param 5 ✓

[5, 256] predictions per token

3 lines that create sparsity

Line 1: x_sparse = ReLU(x @ encoder)

ReLU zeros out all negative values → ~50% sparse

50%

Line 2: y_sparse = ReLU(a_star @ encoder_v)

Another ReLU on the attention output → ~50% sparse

50%

Line 3: gate = x_sparse * y_sparse

ELEMENT-WISE MULTIPLY: output is non-zero ONLY where BOTH are non-zero

50% × 50% = 25% → after training converges to ~5%

~5%

Superposition: Why Transformers Are Hard to Interpret

Transformer (d=256)

Small, dense

256 dimensions

ALL active, ALL dense

What does dim_42 mean?

currency + temperature + verb_tense
+ sentiment + plural + ...
SUPERPOSITION: many concepts!

BDH (n=32,768)

Large, sparse

32,768 neurons

Only ~5% active

What does neuron_7234 mean?

currency
MONOSEMANTIC: one concept!

The Math

10k concepts + 256 dims = 40 concepts/dim (superposition)

10k concepts + 32k neurons = 0.3 concepts/neuron (room to spare!)

Why sparsity is required

32k neurons all active = too expensive. 5% active = 1,600 = manageable

Side-by-Side: Transformer vs BDH Layer

Line	Transformer	BDH	Effect
Q,K,V	3 separate projections	Q=K=RoPE(x_sparse), V=x	Fewer params, tied representations
After projection	Dense (no ReLU)	ReLU(x @ encoder)	Creates sparsity
After attention	Dense	ReLU(a_star @ encoder_v)	More sparsity
Combining	Just add/concat	x_sparse * y_sparse	Multiplicative gating → 5%
Dimension	Stay at d	d → n → d	Expansion enables monosemanticity

Inference

$$Q @ (K^T @ V)$$

↓

p

$$a_star = Q @ p$$

$$p = k[0] \otimes v[0] + k[1] \otimes v[1] + \dots$$

outer product

The Linear Trick: Associativity Without Softmax

Here's where removing softmax matters:

Transformer (with softmax):

```
python
weights = softmax(Q @ K.T) # MUST compute T×T first
output = weights @ V       # Then multiply by V
# Can't reorder - softmax breaks associativity
```

BDH (no softmax):

```
python
output = Q @ K.T @ V       # No softmax!
# Associativity: Q @ (K.T @ V) = (Q @ K.T) @ V
```

Since there's no softmax, we can choose which multiplication to do first.

Regular dot product – two vectors → one number:

$$[1, 2, 3] \cdot [4, 5, 6] = 1 \times 4 + 2 \times 5 + 3 \times 6 = 32 \quad (\text{scalar})$$

Outer product – two vectors → a matrix:

$$[1, 2, 3] \otimes [4, 5, 6] = \text{every combination multiplied}$$

	4	5	6
1	[4,	5,	6]
2	[8,	10,	12]
3	[12,	15,	18]

shape: [3, 3]

In BDH:

k has shape [N=8192]

v has shape [D=256]

outer(k, v) has shape [N, D] = [8192, 256]

Why this helps at inference

Instead of computing the full matrix and storing all past K and V:

Transformer (KV cache grows)

cache_K = [k[0], k[1], k[2], ...] grows with every token

cache_V = [v[0], v[1], v[2], ...] grows with every token

a_star[t] = q[t] @ cache_K.T @ cache_V

BDH does:

BDH (p stays fixed size)

p = 0

see token 0:	a_star[0] = q[0] @ p	← query (p empty, gets nothing)
	p += outer(k[0], v[0])	← write token 0 into memory
see token 1:	a_star[1] = q[1] @ p	← query (sees token 0)
	p += outer(k[1], v[1])	← write token 1 into memory
see token 2:	a_star[2] = q[2] @ p	← query (sees tokens 0 and 1)
	p += outer(k[2], v[2])	← write token 2 into memory

Notice:

- query always happens **before** update – so token t only sees past tokens (causal mask)
- p is always [N, D] – doesn't matter if you've seen 5 tokens or 5 million

Goal for the demo

1. Show a TRAINED BDH model (trained on Wikipedia)
2. Feed it NEW facts it has NEVER seen:
"The CEO of NeuraCorp is Alice Zhang"
"The capital of Zephyria is Windholm"
(Made-up facts — impossible to memorize from training)
3. Ask questions:
"Who is the CEO of NeuraCorp?" → "Alice Zhang"
✓
4. Show Transformer baseline FAILING the same task
(It can't learn without fine-tuning)
5. BONUS: Visualize which synapses strengthened
"Synapse connecting 'CEO' neuron to 'NeuraCorp' neuron increased by 0.34"

→ Build simple streamlit app

→ github repo

→ video

→ make the architecture
train it till it works well

TODO

- pick a dataset (ex: wikipedia)
- setup gcloud (for Hiritesh)
- train 1-2 models
- work on the streamlit app